

SPEC-004 - Login, Autenticación y Backend de Seguridad ROCA.Emuna360

1. Objetivo

Implementar el backend funcional para el manejo de Login y autenticación de usuarios en ROCA.Emuna360, respetando la arquitectura actual de la solución, Arquitectura Limpia, Arquitectura Cebolla, .NET 9, Dapper, ADO.NET y SQL Server.

Esta SPEC debe implementar todo lo relacionado con autenticación desde backend, incluyendo:

- CRUD relacionado con Registro y Usuario.
- Login por correo o documento.
- Validación por DenominacionId.
- Confirmación de correo.
- Activación de usuario.
- Manejo de contraseña segura.
- Generación de token JWT.
- Manejo de refresh token.
- Uso de TokensRefresh.
- Uso de TokensVerificacionCorreo.
- AuthRepository.
- AuthService.
- AuthController.
- DTOs.
- Mappers.
- Result Pattern.
- Manejo de errores controlado.

La implementación debe basarse en las tablas existentes relacionadas con registros, usuarios, roles, menús, tokens y seguridad.

2. Contexto actual

La solución ROCA.Emuna360 ya cuenta con:

- Arquitectura Cebolla.
- Arquitectura Limpia.
- Proyecto Domain.
- Proyecto Application.
- Proyecto Infrastructure.
- Proyecto API.
- Proyecto WebUI.
- Repositorios base.

- Servicios de Application.
- Mappers.
- Result Pattern.
- Manejo global de errores.
- Swagger.
- SQL Server.
- Dapper con ADO.NET.
- Sin Entity Framework.
- Sin DbContext.

3. Tablas involucradas

Las tablas principales para esta SPEC son:

```
Registro
Usuarios
UsuarioRoles
Roles
UsuariosIglesias
Menu_Usuario
Menu_rol
Menu
TokensRefresh
TokensVerificacionCorreo
```

Importante:

```
UsuariosPerfil no se debe usar para esta SPEC.
Los datos personales deben tomarse desde Registro.
Un usuario no puede existir sin un Registro asociado.
Usuarios.RegistroId es obligatorio.
```

3.1 Tabla: Registro

```
CREATE TABLE [dbo].[Registro](
    [RegistroId] [int] IDENTITY(1,1) NOT NULL,
    [DenominacionId] [int] NULL,
    [IglesiaId] [int] NOT NULL,
    [Nombres] [nvarchar](250) NOT NULL,
    [Apellidos] [nvarchar](250) NOT NULL,
    [ParametroIdTipoDocumento] [int] NULL,
    [Documento] [nvarchar](50) NOT NULL,
    [PaisId] [int] NULL,
```

```

[DepartamentoId] [int] NULL,
[CiudadId] [int] NULL,
[CorregimientoId] [int] NULL,
[Direccion] [nvarchar](550) NOT NULL,
[Correo] [nvarchar](500) NULL,
[Telefono] [nvarchar](20) NULL,
[ParametroIdSexo] [int] NOT NULL,
CONSTRAINT [PK_Registro] PRIMARY KEY CLUSTERED
(
    [RegistroId] ASC
)
)

```

Campos

Campo	Tipo	Null	Key	Descripción
RegistroId	int identity	No	PK	Identificador del registro/persona
DenominacionId	int	Sí	FK	Denominación asociada
IglesiaId	int	No	FK	Iglesia asociada
Nombres	nvarchar(250)	No		Nombres de la persona
Apellidos	nvarchar(250)	No		Apellidos de la persona
ParametroIdTipoDocumento	int	Sí		Tipo de documento
Documento	nvarchar(50)	No		Número de documento
PaisId	int	Sí		País
Departamentoid	int	Sí		Departamento
CiudadId	int	Sí		Ciudad
Corregimientoid	int	Sí		Corregimiento
Direccion	nvarchar(550)	No		Dirección
Correo	nvarchar(500)	Sí		Correo personal
Telefono	nvarchar(20)	Sí		Teléfono
ParametroIdSexo	int	No		Sexo

Llaves foráneas

```

FK_Registro_Denominaciones → Denominaciones(DenominacionId)
FK_Registro_Iglesias → Iglesias(IglesiaId)

```

3.2 Tabla: Usuarios

```
CREATE TABLE [dbo].[Usuarios](
    [UsuarioId] [int] IDENTITY(1,1) NOT NULL,
    [DenominacionId] [int] NOT NULL,
    [RegistroId] [int] NOT NULL,
    [Correo] [nvarchar](150) NOT NULL,
    [PasswordHash] [nvarchar](500) NOT NULL,
    [EmailVerificado] [bit] NOT NULL,
    [Bloqueado] [bit] NOT NULL,
    [FechaCreacion] [datetime2](0) NOT NULL,
    [UltimoLogin] [datetime2](0) NULL,
    [SecurityStamp] [nvarchar](64) NOT NULL,
    [RolId] [int] NOT NULL,
    CONSTRAINT [PK_Usuarios] PRIMARY KEY CLUSTERED
(
    [UsuarioId] ASC
),
    CONSTRAINT [UQ_Usuarios_Correo] UNIQUE NONCLUSTERED
(
    [Correo] ASC
)
)
```

Campos

Campo	Tipo	Null	Key	Descripción
Usuariold	int identity	No	PK	Identificador del usuario
DenominacionId	int	No	FK	Denominación asociada
RegistroId	int	No	FK	Registro/persona asociada
Correo	nvarchar(150)	No	UQ	Correo de autenticación
PasswordHash	nvarchar(500)	No		Hash de contraseña
EmailVerificado	bit	No		Indica si confirmó correo
Bloqueado	bit	No		Indica si está bloqueado
FechaCreacion	datetime2(0)	No		Fecha de creación
UltimoLogin	datetime2(0)	Sí		Fecha último login
SecurityStamp	nvarchar(64)	No		Sello de seguridad
RolId	int	No	FK	Rol principal

Defaults

```
DF_Usuarios_EmailVerificado = 0
DF_Usuarios_Bloqueado = 0
DF_Usuarios_FechaCreacion = sysdatetime()
DF_Usuarios_SecurityStamp = newid()
```

Llaves foráneas

```
FK_Usuarios_Denominaciones → Denominaciones(DenominacionId)
FK_Usuarios_Registro → Registro(RegistroId)
FK_Usuarios_Roles → Roles(RolId)
```

Reglas importantes

Un Usuario siempre debe estar asociado a un Registro.
No se debe crear Usuario sin RegistroId válido.
Correo de Usuario se usa para autenticación por correo.
Documento para login debe buscarse desde Registro.Documento.
Datos personales no viven en Usuarios, viven en Registro.

3.3 Tabla: UsuarioRoles

```
CREATE TABLE [dbo].[UsuarioRoles](
    [UsuarioRolId] [int] IDENTITY(1,1) NOT NULL,
    [UsuarioId] [int] NULL,
    [DenominacionId] [int] NULL,
    [RolId] [int] NULL,
    [FechaAsignacion] [datetime2](0) NOT NULL,
    CONSTRAINT [PK_UsuarioRoles] PRIMARY KEY CLUSTERED
(
    [UsuarioRolId] ASC
)
)
```

Campos

Campo	Tipo	Null	Key
UsuarioRolId	int identity	No	PK
UsuarioId	int	Sí	FK
DenominacionId	int	Sí	FK

Campo	Tipo	Null	Key
RolId	int	Sí	FK
FechaAsignacion	datetime2(0)	No	

Llaves foráneas

```
FK_UsuarioRoles_Denominaciones → Denominaciones(DenominacionId)
FK_UsuarioRoles_Roles → Roles(RolId)
FK_UsuarioRoles_Usuarios → Usuarios(UsuarioId)
```

3.4 Tabla: Roles

```
CREATE TABLE [dbo].[Roles](
    [RolId] [int] IDENTITY(1,1) NOT NULL,
    [DenominacionId] [int] NULL,
    [Nombre] [nvarchar](100) NOT NULL,
    [Codigo] [nvarchar](50) NOT NULL,
    [Activo] [bit] NOT NULL,
    [FechaCreacion] [datetime2](0) NOT NULL,
    CONSTRAINT [PK_Roles] PRIMARY KEY CLUSTERED
(
    [RolId] ASC
)
```

Campos

Campo	Tipo	Null	Key
RolId	int identity	No	PK
DenominacionId	int	Sí	FK
Nombre	nvarchar(100)	No	
Codigo	nvarchar(50)	No	
Activo	bit	No	
FechaCreacion	datetime2(0)	No	

Llaves foráneas

```
FK_Roles_Denominaciones → Denominaciones(DenominacionId)
```

3.5 Tabla: UsuariosIglesias

```
CREATE TABLE [dbo].[UsuariosIglesias](
    [UsuarioIglesiaId] [int] NOT NULL,
    [DenominacionId] [int] NOT NULL,
    [UsuarioId] [int] NOT NULL,
    [IglesiaId] [int] NOT NULL,
    [Estado] [tinyint] NOT NULL,
    [EsAdministrador] [bit] NOT NULL,
    [FechaCreacion] [datetime2](0) NOT NULL
)
```

Campos

Campo	Tipo	Null	Key
Usuariolglesiald	int	No	
DenominacionId	int	No	FK
UsuarioId	int	No	FK
Iglesiald	int	No	FK
Estado	tinyint	No	
EsAdministrador	bit	No	
FechaCreacion	datetime2(0)	No	

Llaves foráneas

```
FK_UsuariosIglesias_Denominaciones → Denominaciones(DenominacionId)
FK_UsuariosIglesias_Iglesias → Iglesias(IglesiaId)
FK_UsuariosIglesias_Usuarios → Usuarios(UsuarioId)
```

3.6 Tabla: Menu_Usuario

```
CREATE TABLE [dbo].[Menu_Usuario](
    [OpcUsuId] [int] IDENTITY(1,1) NOT NULL,
    [DenominacionId] [int] NOT NULL,
    [UsuarioId] [int] NOT NULL,
    [MenuId] [int] NOT NULL,
    CONSTRAINT [PK_Menu_Usuario] PRIMARY KEY CLUSTERED
(
    [OpcUsuId] ASC
)
```

```
)  
)
```

Campos

Campo	Tipo	Null	Key
OpcUsuld	int identity	No	PK
DenominacionId	int	No	FK
Usuariold	int	No	
MenuId	int	No	

Llaves foráneas

```
FK_Menu_Usuario_Denominaciones → Denominaciones(DenominacionId)
```

3.7 Tabla: Menu_rol

```
CREATE TABLE [dbo].[Menu_rol](  
    [Menu_Rol] [int] IDENTITY(1,1) NOT NULL,  
    [DenominacionId] [int] NOT NULL,  
    [RolId] [int] NOT NULL,  
    [MenuId] [int] NOT NULL,  
    [Estado] [bit] NOT NULL,  
    CONSTRAINT [PK_Menu_rol] PRIMARY KEY CLUSTERED  
(  
    [Menu_Rol] ASC  
)  
)
```

Campos

Campo	Tipo	Null	Key
Menu_Rol	int identity	No	PK
DenominacionId	int	No	FK
RolId	int	No	
MenuId	int	No	
Estado	bit	No	

Llaves foráneas


```
FK_Menu_rol_Denominaciones → Denominaciones(DenominacionId)
```

3.8 Tabla: Menu

```
CREATE TABLE [dbo].[Menu](
    [MenuId] [int] NULL,
    [DenominacionId] [int] NULL,
    [Descripcion] [nvarchar](150) NOT NULL,
    [IdGrupo] [int] NULL,
    [Tipo] [nchar](1) NOT NULL,
    [Estado] [bit] NOT NULL,
    [Url] [nvarchar](500) NOT NULL,
    [Imagen] [nvarchar](500) NOT NULL,
    [Fecha] [datetime] NOT NULL,
    [Orden] [int] NOT NULL,
    [NuevaVentana] [bit] NOT NULL
)
```

Campos

Campo	Tipo	Null	Key
MenuId	int	Sí	
DenominacionId	int	Sí	FK
Descripcion	nvarchar(150)	No	
IdGrupo	int	Sí	
Tipo	nchar(1)	No	
Estado	bit	No	
Url	nvarchar(500)	No	
Imagen	nvarchar(500)	No	
Fecha	datetime	No	
Orden	int	No	
NuevaVentana	bit	No	

Llaves foráneas

```
FK_Menu_Denominaciones → Denominaciones(DenominacionId)
```

3.9 Tabla: TokensRefresh

```
CREATE TABLE [dbo].[TokensRefresh](
    [RefreshTokenId] [int] NOT NULL,
    [DenominacionId] [int] NOT NULL,
    [UsuarioId] [int] NOT NULL,
    [IglesiaId] [int] NOT NULL,
    [TokenHash] [nvarchar](128) NOT NULL,
    [ExpiraEn] [datetime2](0) NOT NULL,
    [RevocadoEn] [datetime2](0) NULL,
    [ReemplazadoPor] [int] NULL,
    [UserAgent] [nvarchar](256) NULL,
    [Ip] [nvarchar](45) NULL,
    [FechaCreacion] [datetime2](0) NOT NULL,
    CONSTRAINT [PK_TokensRefresh] PRIMARY KEY CLUSTERED
(
    [RefreshTokenId] ASC
)
)
```

Campos

Campo	Tipo	Null	Key
RefreshTokenId	int	No	PK
DenominacionId	int	No	FK
UsuarioId	int	No	
Iglesiald	int	No	
TokenHash	nvarchar(128)	No	
ExpiraEn	datetime2(0)	No	
RevocadoEn	datetime2(0)	Sí	
ReemplazadoPor	int	Sí	
UserAgent	nvarchar(256)	Sí	
Ip	nvarchar(45)	Sí	
FechaCreacion	datetime2(0)	No	

Llaves foráneas

```
FK_TokensRefresh_Denominaciones → Denominaciones(DenominacionId)
```

3.10 Tabla: TokensVerificacionCorreo

```
CREATE TABLE [dbo].[TokensVerificacionCorreo](
    [TokenId] [int] IDENTITY(1,1) NOT NULL,
    [DenominacionId] [int] NULL,
    [UsuarioId] [int] NULL,
    [IglesiaId] [int] NULL,
    [TokenHash] [nvarchar](128) NOT NULL,
    [ExpiraEn] [datetime2](0) NOT NULL,
    [UsadoEn] [datetime2](0) NULL,
    [FechaCreacion] [datetime2](0) NOT NULL,
    CONSTRAINT [PK_TokensVerificacionCorreo] PRIMARY KEY CLUSTERED
(
    [TokenId] ASC
)
```

Campos

Campo	Tipo	Null	Key
TokenId	int identity	No	PK
DenominacionId	int	Sí	FK
UsuarioId	int	Sí	
IglesiaId	int	Sí	
TokenHash	nvarchar(128)	No	
ExpiraEn	datetime2(0)	No	
UsadoEn	datetime2(0)	Sí	
FechaCreacion	datetime2(0)	No	

Llaves foráneas

FK_TokensVerificacionCorreo_Denominaciones → Denominaciones(DenominacionId)

4. Reglas obligatorias

1. No usar Entity Framework.
2. No crear DbContext.
3. Usar Dapper con ADO.NET.
4. Usar procedimientos almacenados.

5. No usar SQL inline como implementación final.
6. No usar SELECT *.
7. No exponer PasswordHash.
8. No exponer TokenHash.
9. No exponer stacktrace.
10. Usar Result Pattern.
11. Usar DTOs.
12. Usar Entities correctamente.
13. Usar Mappers correctamente.
14. API no debe consumir repositorios directamente.
15. API debe consumir AuthService.
16. AuthService debe consumir IAuthRepository.
17. SQL solo en Infrastructure.
18. Login debe validar DenominacionId.
19. Login debe permitir correo o documento.
20. Documento debe consultarse desde Registro.Documento.
21. Correo para login debe consultarse desde Usuarios.Correo.
22. Datos personales deben consultarse desde Registro.
23. Usuario debe tener RegistroId obligatorio.
24. No se debe usar UsuariosPerfil en esta SPEC.
25. Debe existir manejo de correo de verificación.
26. Debe existir manejo de RefreshToken.
27. Debe existir manejo de TokensVerificacionCorreo.
28. Mantener .NET 9.
29. Mantener Arquitectura Limpia.
30. Mantener Arquitectura Cebolla.

5. Flujo arquitectónico obligatorio

```
AuthController
→ IAuthService
→ AuthService
→ Mapper
→ Domain Entity
→ IAuthRepository
→ AuthRepository
→ Stored Procedures
→ SQL Server
```

6. Funcionalidades requeridas

6.1 Login

Debe permitir:

```
Correo + Contraseña + DenominacionId
Documento + Contraseña + DenominacionId
```

Debe validar:

- DenominacionId.
- Usuario existente.
- Usuario asociado a Registro.
- Usuario no bloqueado.
- Email verificado.
- PasswordHash.
- Rol principal.
- Roles adicionales si existen.
- Iglesia asociada.
- Menús asociados si aplica.

Reglas:

- Si el login contiene correo, buscar por Usuarios.Correo.
- Si el login contiene documento, buscar por Registro.Documento.
- La búsqueda por documento debe unir Usuarios con Registro por RegistroId.
- Siempre se debe filtrar por DenominacionId.
- No se debe consultar UsuariosPerfil.
- Si no existe usuario, devolver error controlado.
- Si no existe Registro asociado, devolver error controlado.
- Si el usuario está bloqueado, devolver error controlado.
- Si EmailVerificado es false, devolver error controlado.
- Si la contraseña es incorrecta, devolver error controlado.
- Si todo es correcto, generar JWT y RefreshToken.
- Actualizar UltimoLogin.

6.2 Registro / creación de usuario

Debe crear usuario únicamente si existe o se crea un Registro válido.

Flujo esperado:

```
Validar DenominacionId
Validar IglesiaId
Validar datos de Registro
Validar Documento
Validar Correo
Crear Registro si no existe
Crear Usuario asociado al RegistroId
Asignar Rol
```

```
Asignar Iglesia
Crear token de verificación de correo
Enviar correo de confirmación
```

Reglas:

- No se puede crear Usuario sin RegistroId.
- Registro debe pertenecer a la misma DenominacionId.
- Registro debe tener IglesiaId.
- Documento debe estar en Registro.
- Datos personales deben estar en Registro.
- Correo del usuario debe estar en Usuarios.Correo.
- Correo de Registro puede usarse como correo personal.
- EmailVerificado debe iniciar en false.
- Bloqueado debe iniciar en false.
- PasswordHash debe guardarse seguro.
- SecurityStamp debe generarse.
- Debe evitar duplicados por correo.
- Debe evitar duplicados por documento dentro de DenominacionId.
- Debe manejar concurrencia con transacción.

6.3 Confirmación de correo

Debe permitir confirmar correo usando token.

Reglas:

- Buscar token en TokensVerificacionCorreo.
- Validar hash del token.
- Validar DenominacionId.
- Validar UsuarioId.
- Validar IglesiaId si aplica.
- Validar que no esté vencido.
- Validar que no esté usado.
- Marcar token como usado.
- Marcar Usuarios.EmailVerificado = true.
- Retornar resultado controlado.

6.4 Refresh Token

Debe permitir renovar token usando RefreshToken.

Reglas:

- Validar hash del refresh token.
- Validar DenominacionId.
- Validar UsuarioId.

- Validar Iglesiald.
- Validar expiración.
- Validar que no esté revocado.
- Revocar token anterior.
- Crear nuevo refresh token.
- Guardar hash del nuevo refresh token.
- Generar nuevo JWT.
- Retornar nuevo par de tokens.

6.5 Logout

Debe permitir cerrar sesión.

Reglas:

- Revocar refresh token actual.
- Guardar RevocadoEn.
- No eliminar físicamente el token.

7. DTOs requeridos

Ubicación:

```
ROCA.Emuna360.Application/DTOs/Auth
```

Crear:

```
LoginRequestDto  
LoginResponseDto  
RegisterUserRequestDto  
RegisterUserResponseDto  
ConfirmEmailRequestDto  
RefreshTokenRequestDto  
RefreshTokenResponseDto  
LogoutRequestDto  
AuthUserDto  
AuthRoleDto  
AuthMenuDto  
AuthRegistroDto
```

LoginRequestDto

Debe incluir:

```
DenominacionId
Login
Password
IglesiaId opcional
```

RegisterUserRequestDto

Debe incluir datos para:

```
DenominacionId
IglesiaId
Nombres
Apellidos
ParametroIdTipoDocumento
Documento
PaisId
DepartamentoId
CiudadId
CorregimientoId
Direccion
Correo
Telefono
ParametroIdSexo
Password
RolId
```

8. Entidades Domain relacionadas

Validar o crear si no existen:

```
Registro
Usuario
Rol
UsuarioRol
UsuarioIglesia
Menu
MenuRol
MenuUsuario
RefreshToken
TokenVerificacionCorreo
AuthUser
AuthRole
AuthMenu
```


9. Interfaces requeridas

Repositories

Ubicación:

```
ROCA.Emuna360.Application/Interfaces/Repositories
```

Crear:

```
IAuthRepository
```

Services

Ubicación:

```
ROCA.Emuna360.Application/Interfaces/Services
```

Crear:

```
IAuthService  
IPasswordHasherService  
IJwtTokenService  
IEmailSenderService  
IRefreshTokenService  
IVerificationTokenService
```

10. Servicios requeridos

Ubicación:

```
ROCA.Emuna360.Application/Services/Auth
```

Crear:

```
AuthService  
PasswordHasherService
```

```
RefreshTokenService  
VerificationTokenService
```

JWT

Ubicación sugerida:

```
ROCA.Emuna360.Infrastructure/Security
```

Crear:

```
JwtTokenService
```

Correo

Ubicación sugerida:

```
ROCA.Emuna360.Infrastructure/Services/Email
```

Crear:

```
EmailSenderService
```

11. Repositorio requerido

Crear:

```
ROCA.Emuna360.Infrastructure/Repositories/Security/AuthRepository.cs
```

Debe implementar:

```
IAuthRepository
```

Debe usar:

- Dapper.
- ADO.NET.

- Stored Procedures.
- DynamicParameters.
- CommandType.StoredProcedure.
- Transacciones cuando aplique.

12. Métodos mínimos de IAuthRepository

```
Task<AuthUser?> GetUserByEmailAsync(int denominacionId, string correo);
Task<AuthUser?> GetUserByDocumentAsync(int denominacionId, string documento);
Task<Registro?> GetRegistroByDocumentoAsync(int denominacionId, string documento);
Task<Registro?> GetRegistroByIdAsync(int denominacionId, int registroId);
Task<int> CreateRegistroAsync(Registro entity);
Task<int> CreateUserAsync(Usuario entity);
Task<bool> AssignUserRoleAsync(int denominacionId, int usuarioId, int rolId);
Task<bool> AssignUserIglesiaAsync(int denominacionId, int usuarioId, int
iglesiaId, bool esAdministrador);
Task<bool> UpdateLastLoginAsync(int denominacionId, int usuarioId);
Task<bool> MarkEmailAsVerifiedAsync(int denominacionId, int usuarioId);
Task<int> CreateEmailVerificationTokenAsync(TokenVerificacionCorreo entity);
Task<TokenVerificacionCorreo?> GetEmailVerificationTokenAsync(int denominacionId,
string tokenHash);
Task<bool> MarkEmailVerificationTokenAsUsedAsync(int denominacionId, int tokenId);
Task<int> CreateRefreshTokenAsync(RefreshToken entity);
Task<RefreshToken?> GetRefreshTokenAsync(int denominacionId, string tokenHash);
Task<bool> RevokeRefreshTokenAsync(int denominacionId, int refreshTokenId, int?
reemplazadoPor);
Task<IEnumerable<AuthRole>> GetUserRolesAsync(int denominacionId, int usuarioId);
Task<IEnumerable<AuthMenu>> GetUserMenusAsync(int denominacionId, int usuarioId);
```

13. Procedimientos almacenados requeridos

Si ya existen, reutilizarlos.

Si no existen, dejar documentado el nombre requerido y preparar el repositorio para consumirlos.

```
sp_auth_obtener_usuario_por_correo
sp_auth_obtener_usuario_por_documento
sp_auth_obtener_registro_por_documento
sp_auth_obtener_registro_por_id
sp_auth_crear_registro
sp_auth_crear_usuario
sp_auth_asignar_usuario_rol
sp_auth_asignar_usuario_iglesia
sp_auth_actualizar_ultimo_login
sp_auth_marcar_email_verificado
```

```
sp_auth_crear_token_verificacion_correo
sp_auth_obtener_token_verificacion_correo
sp_auth_marcar_token_verificacion_usado
sp_auth_crear_refresh_token
sp_auth_obtener_refresh_token
sp_auth_revocar_refresh_token
sp_auth_obtener_rol_usuario
sp_auth_obtener_menu_usuario
```

Si existe `usp_Obtener_Auth`, debe analizarse para determinar si puede reutilizarse en login.

Importante:

Todo SP de Auth debe recibir DenominacionId cuando aplique.
Los SP de creación deben garantizar transacción y evitar datos parciales.
No se deben crear SP físicamente desde esta SPEC salvo que se indique explícitamente en una SPEC de base de datos.

14. AuthController

Crear:

```
ROCA.Emuna360.API/Controllers/Auth/AuthController.cs
```

Endpoints:

```
POST api/v1/auth/login
POST api/v1/auth/register
POST api/v1/auth/confirm-email
POST api/v1/auth/refresh-token
POST api/v1/auth/logout
GET  api/v1/auth/me
```

El controlador debe:

- Inyectar `IAuthService`.
- No inyectar repositorios.
- No usar SQL.
- No usar Dapper.
- Retornar `ApiResponse`.
- Usar Result Pattern.
- Manejar errores controlados.
- No exponer entidades directamente.

- No exponer PasswordHash.
- No exponer TokenHash.

15. JWT

Agregar configuración:

```
"Jwt": {  
  "Issuer": "ROCA.Emuna360",  
  "Audience": "ROCA.Emuna360.Client",  
  "SecretKey": "CAMBIAR_POR_CLAVE_SEGURA",  
  "AccessTokenMinutes": 30,  
  "RefreshTokenDays": 30  
}
```

Crear configuración tipada:

```
JwtSettings
```

Claims mínimos:

```
UsuarioId  
RegistroId  
DenominacionId  
IglesiaId  
Correo  
Documento  
NombreCompleto  
RolId  
Roles  
SecurityStamp
```

16. PasswordHasher

Debe existir:

```
IPasswordHasherService
```

Métodos:

```
string HashPassword(string password);  
bool VerifyPassword(string password, string passwordHash);
```

No usar:

```
MD5  
SHA1  
SHA256 simple sin salt  
Contraseña plana
```

17. TokensRefresh

Debe:

- Guardar hash.
- Asociar DenominacionId.
- Asociar UsuarioId.
- Asociar IglesiaId.
- Guardar expiración.
- Guardar revocación.
- Guardar UserAgent.
- Guardar IP.
- Manejar reemplazo.

Nunca guardar token plano.

18. TokensVerificacionCorreo

Debe:

- Guardar hash.
- Asociar DenominacionId.
- Asociar UsuarioId.
- Asociar IglesiaId.
- Manejar expiración.
- Manejar uso.
- Confirmar correo.

Nunca guardar token plano.

19. Concurrencia

Debe:

- Usar transacciones.
 - Evitar duplicados por Usuarios.Correo.
 - Evitar duplicados por Registro.Documento dentro de DenominacionId.
 - Validar RegistroId.
 - Validar DenominacionId.
 - Validar Iglesiald.
 - Evitar creación parcial.
 - Manejar errores controlados.
 - Usar stored procedures transaccionales.
-

20. Correo

Debe existir:

```
IEmailSenderService
```

Método:

```
Task SendEmailConfirmationAsync(string email, string fullName, string  
confirmationUrl);
```

No quemar credenciales en código.

21. Resultados esperados

Debe existir:

- AuthController.
- IAuthService.
- AuthService.
- IAuthRepository.
- AuthRepository.
- DTOs Auth.
- Mappers Auth.
- PasswordHasherService.
- JwtTokenService.
- RefreshTokenService.

- VerificationTokenService.
 - EmailSenderService.
 - Configuración JWT.
 - Configuración DI.
 - Login funcional por correo.
 - Login funcional por documento.
 - Confirmación de correo preparada.
 - Refresh token preparado.
 - Logout preparado.
 - Swagger funcionando.
-

22. Criterios de aceptación

- La solución compila.
 - La API inicia.
 - Swagger carga.
 - Login por correo funciona.
 - Login por documento funciona.
 - Login valida DenominacionId.
 - Login usa Registro.Documento para documento.
 - Login usa Usuarios.Correo para correo.
 - Usuario requiere RegistroId.
 - Usuario no se crea sin Registro.
 - UsuariosPerfil no se usa.
 - Login rechaza usuario bloqueado.
 - Login rechaza correo no verificado.
 - Login valida contraseña segura.
 - Login genera JWT.
 - Login genera RefreshToken.
 - RefreshToken se guarda como hash.
 - Token de verificación se guarda como hash.
 - Confirmación de correo activa usuario.
 - Logout revoca refresh token.
 - API no consume repositorios directamente.
 - Application no contiene SQL.
 - Infrastructure contiene acceso a datos.
 - No existe Entity Framework.
 - No existe DbContext.
-

23. Fase 1 - Análisis

Antes de implementar, AntigraVity debe analizar:

1. Arquitectura actual.

2. Tablas relacionadas.
3. Entidades existentes.
4. DTOs existentes.
5. Mappers existentes.
6. Result Pattern.
7. Repositorios existentes.
8. Servicios existentes.
9. Si UsuariosPerfil está siendo usado incorrectamente.
10. Si Usuario ya contiene Registroid.
11. Si existe `usp_Obtener_Auth`.
12. Qué procedimientos se pueden reutilizar.
13. Qué procedimientos faltan.
14. Qué archivos se deben crear.
15. Qué archivos se deben modificar.
16. Riesgos técnicos.
17. Plan de implementación.

No implementar nada hasta aprobar el análisis.

24. Fase 2 - Implementación

Solo después de aprobar análisis:

1. Crear DTOs Auth.
2. Crear o ajustar entidades faltantes.
3. Crear Mappers Auth.
4. Crear IAuthRepository.
5. Crear AuthRepository.
6. Crear IAuthService.
7. Crear AuthService.
8. Crear PasswordHasherService.
9. Crear JwtTokenService.
10. Crear RefreshTokenService.
11. Crear VerificationTokenService.
12. Crear EmailSenderService.
13. Crear AuthController.
14. Configurar DI.
15. Configurar JWT.
16. Configurar Swagger si aplica.
17. Compilar.
18. Corregir errores.
19. Probar endpoints.